In this example, we're taking a simpler approach. Instead of integrating with a graph service, we provide an `entities.json` file with each request to AVP. This shows how policies and data can be integrated easily without needing ACME's full infrastructure. Usually, you would generate these JSON snippets dynamically—one for each request—by querying the relationship graph, not by editing files manually.

A.6 Evaluating authorization requests

Once the schema, policies, and entities are set up, the authorization system can then answer real questions. In Cedar and AVP, this is done via the `is-authorized` API. Each call provides three inputs: the principal (who is making the request), the action they want to perform, and the resource involved. Additionally, a context object can be included to represent environment attributes, such as the device used or the time of day. The policy engine assesses these inputs against the stored policies and entities and returns either `allow` or `deny`.

A.6.1 Owner actions

Since Alice is the document owner, the owner-baseline policy applies to all document actions, not just viewing. Whether the request is to view, edit, or share, the policy checks `if resource.owner == principal` and allows the action. This highlights one of the main benefits of writing a baseline policy for owners: It prevents duplicate logic across separate view/edit/share policies and ensures consistency in how ownership is enforced:

```
aws verifiedpermissions is-authorized \
  --policy-store-id "$STORE_ID" \
  --principal entityType=ACME::Employee,entityId=alice \
  --action actionType=ACME::Action,actionId=doc:view \
  --resource entityType=ACME::Document,entityId=q3-plan \
  --context '{"contextMap":
     {"device":{"record":{"managed":{"boolean":true}}}}}' \
  --entities file://acme-entities.json
```

The result of this call is the following JSON: